🚀 C Programming: The Foundations & Hidden Traps

This guide summarizes the critical "PGEE Traps" and core behaviors of C programming, focusing on data types, operators, and memory.

💎 1. Data Types & Memory Layout

In C, memory allocation is fixed based on the type. A key concept is that literals (constant values) have default types.

Type	Size (Typical)	Key Characteristic
char	1 Byte	Stores small integers or ASCII values (-128 to 127).
int	4 Bytes	Stores whole numbers.
float	4 Bytes	Single precision decimal (6 decimal places).
double	8 Bytes	**Default for decimals** (e.g., 3.14 is a double).

⚠️ The sizeof() Trap

- sizeof(3.14) returns **8** (because it's a double).
- sizeof(3.14f) returns **4** (the f suffix forces it to be a float).
- sizeof('A') in C can return **4** in some compilers because character literals are promoted to int during evaluation!

🪓 2. Arithmetic & Integer Division

The most common logic error in C exams.

- **Rule:** int / int = int (The decimal part is **truncated/cut off**, not rounded).
- **Example:** 5 / 2 results in 2.
- **The Fix:** At least one operand must be a float: 5.0 / 2 results in 2.5.

Type Casting

- **Implicit:** C automatically promotes smaller types (e.g., int + float results in a float).

- **Explicit:** Manually forcing a type: (float)5 / 2 results in 2.5.

🔡 3. Characters & ASCII Math

Characters in C are simply integers disguised as symbols.

- 'A' = 65, 'B' = 66, etc.
- **Math on Chars:** 'A' + 1 is 66.
- **Printing:**
  - %d prints the number (66).
  - %c prints the character (B).

🔄 4. The Overflow (Wrap-around) Effect

Variables have a "bucket size." If you exceed the maximum value, it wraps around to the minimum.

- **Example (Signed 16-bit):**
  - Max: 32767
  - 32767 + 1 ➡️ -32768 (Wraps to the negative end).
- This is like a circular odometer on a car.

⚡ 5. Logical Operators & Short-Circuiting

C is "lazy" (efficient) when evaluating logical expressions.

- **&& (AND):** If the first part is **False**, C skips the second part.
- **|| (OR):** If the first part is **True**, C skips the second part.
- **Benefit:** This prevents errors like division by zero: (5 > 2) || (10 / 0 == 1) will **not** crash because 5 > 2 is true.

🔢 6. Bitwise & Modulo

- **Modulo (%):** Returns the **remainder**. 10 % 3 = 1.
- **Bitwise AND (&):** Compares numbers bit-by-bit. 5 & 1 (Binary 0101 & 0001) = 1.
- **Bitwise OR (|):** Result is 1 if either bit is 1.

🧠 Self-Check: Predict the Output

Try to solve these in your head before checking the code:

1. printf("%d", 7 / 2); ➡️ **3**
2. printf("%f", 5 / 2.0); ➡️ **2.500000**
3. printf("%d", 'C' - 'A'); ➡️ **2** (67 - 65)
4. int x = 5; printf("%d", x++); ➡️ **5** (Post-increment prints first, then adds).